ML4SCI GSoC 2026 — Task 2b

Super-Resolution at the CMS Detector (Jet Images)

Repository `KenWuqianghao/CMS_E2E`

March 21, 2026

Contents

1 Objective 1

2 Dataset and labels 1

3 Model 1

4 Training and protocols 1

5 Evaluation 2

6 Reference results (checked in) 2

7 Code layout 2

---

1 Objective

Train a **Generative Adversarial Network** to map low-resolution calorimeter jet images to high-resolution targets, using the official ML4SCI parquet release (CERNBox link in the GSoC PDF). The submission documents modeling and optimization choices as required by the task statement.

2 Dataset and labels

- **Low resolution:** `X_jets_LR`, shape $3 \times 64 \times 64$.
- **High resolution:** `X_jets`, shape $3 \times 125 \times 125$.
- **Class label:** `y` $\in \{0, 1\}$ (quark vs. gluon), stratified in train/validation/test splits (default 80%/10%/10%).

The public shards use a **single Parquet row group per file**; the loader therefore **streams record batches** and materializes only the row indices needed for training, avoiding full-file loads per sample.

3 Model

- **Generator (SRGAN-style):** bicubic upsampling of LR to 125×125 as an explicit baseline, plus a residual CNN (residual blocks, InstanceNorm) predicting a correction added to that baseline.

- **Discriminator (PatchGAN):** convolutional stack with optional **class conditioning** via embedding-modulated feature maps (quark/gluon).
- **Losses:** least-squares GAN objectives; **L1** reconstruction on normalized HR; small weight on **adversarial** loss; optional **energy-matching** term on denormalized total deposited energy ($\sum \max(0, x)$ over channels and pixels).

4 Training and protocols

Two tiers are supported in code and documentation:

1. **Host-constrained (reference):** materialized subsets via `-max-train-samples` / `-max-val-samples`, fewer epochs, optional `-memmap-train` when RAM is insufficient for full tensors.
2. **Full-scale:** no subset caps, `-epochs 20`, `-memmap-train` as needed for ∼full training index sets.

Validation logs `val_l1`, bicubic baseline `val_bicubic_l1`, and `val_energy_mae` to `history.csv`. Checkpoints: `checkpoint_best.pt` (best val L1), `checkpoint_last.pt`.

5 Evaluation

Script `super_resolution/evaluate.py` reports JSON metrics on the test split (full test by default), including SR vs. HR and a **bicubic** baseline: PSNR/SSIM, mean total-energy ratio, energy MAE, mean radial-profile L1, and a toy linear probe on per-image channel means. Results are written as structured JSON (e.g. `metrics_report.json`).

6 Reference results (checked in)

Using the **full_run** configuration (subset caps, 8 epochs, $\lambda_{\text{energy}} = 0.05$) on the full test split ($n = 13\,932$), illustrative numbers from `super_resolution/runs/full_run/metrics_report.json`:

Metric	SR	Bicubic
PSNR mean (dB)	44.28	44.28
SSIM mean	0.9935	0.9934
Energy MAE vs. HR	1.20	21.29
Mean radial profile L1 vs. HR	0.00040	0.00132

The energy penalty materially improves **energy fidelity** vs. bicubic; image PSNR/SSIM remain close to the trivial resize because the task geometry is dominated by a smooth upscale.

An **ablation** with `-lambda-energy 0` (`ablation_no_energy/`) shows degraded energy scale (e.g. energy MAE ≈ 153 vs. HR) despite similar PSNR, illustrating the need for physics-aware terms beyond pixel loss.

7 Code layout

`super_resolution/data.py`, `models.py`, `train.py`, `evaluate.py`; narrative in `MODEL_DISCUSSION.md` and `super_resolution/README.md`.

Report artifact

This PDF is built from `docs/GSoC26_Task2b_Report.tex` in `CMS_E2E`.